NH-ATMS 프로젝트 구조 가이드

> 최종 수정: 2026-04-01
> 프리랜서 투입 전 반드시 숙지

---

1. 1. 전체 프로젝트 구조

```
2.프로젝트/
├── 0.기본자료/                        # AS-IS 분석, 참고자료
│   ├── 농협은행/                       # NH AS-IS 메뉴분석, UI 프로토타입
│   └── 새마을금고/                     # 새마을금고 참고 소스, SQL
├── 1.Project/                         # ★ 실제 소스코드
│   ├── Backend/
│   │   └── nh-atms-api/               # Spring Boot API 서버
│   └── Frontend/
│       └── nh-atms-web/               # React SPA
├── 2.개발문서/                        # 개발 가이드 문서
│   ├── 01_코딩컨벤션.md
│   ├── 02_API스펙정의.md
│   ├── 03_화면작업명세서_템플릿.md
│   ├── 04_DB스키마설계.md
│   └── 05_프로젝트구조.md             # (이 파일)
└── readme.md
```

---

2. 2. Backend 구조 (nh-atms-api)

2.1. 2.1 기술 스택

항목	버전	비고
Java	21	LTS
Spring Boot	3.5.0	
MyBatis	3.0.4 (mybatis-spring-boot-starter)	
Spring Security	6.x (Boot 내장)	JWT 기반
jjwt	0.12.6	JWT 토큰 생성/검증
Springdoc OpenAPI	2.8.4	Swagger UI
Lombok	Boot 내장	
Gradle	Wrapper 포함	

2.2. 2.2 디렉토리 구조

```
nh-atms-api/
├── build.gradle                      # 빌드 설정, 의존성
├── settings.gradle                   # 멀티모듈 설정
├── gradlew / gradlew.bat             # Gradle Wrapper
├── src/
│   ├── main/
│   │   ├── java/atms/
│   │   │   ├── NhAtmsApiApplication.java    # @SpringBootApplication
│   │   │   ├── config/                      # 설정
│   │   │   │   └── SecurityConfig.java      # Security + CORS
│   │   │   ├── common/                      # 공통
│   │   │   │   ├── exception/
│   │   │   │   │   ├── BusinessException.java
│   │   │   │   │   └── GlobalExceptionHandler.java
│   │   │   │   └── response/
│   │   │   │       └── ApiResponse.java     # 공통 응답 포맷
│   │   │   ├── auth/                        # ☑ 인증 (구현 완료)
│   │   │   │   ├── controller/
│   │   │   │   │   └── AuthController.java
│   │   │   │   ├── dto/
│   │   │   │   │   ├── LoginRequest.java
│   │   │   │   │   ├── LoginResponse.java
│   │   │   │   │   └── RefreshRequest.java
│   │   │   │   ├── jwt/
│   │   │   │   │   ├── JwtTokenProvider.java
│   │   │   │   │   └── JwtAuthenticationFilter.java
│   │   │   │   ├── service/
│   │   │   │   │   └── AuthService.java
│   │   │   │   └── sso/
│   │   │   │       └── MockSsoService.java
│   │   │   │
│   │   │   ├── {도메인}/                    # ← 비즈니스 도메인 추가
│   │   │   │   ├── controller/
│   │   │   │   ├── dto/
│   │   │   │   ├── service/
│   │   │   │   └── mapper/
│   │   │   └── ...
│   │   └── resources/
│   │       ├── application.yml              # 설정 파일
│   │       └── mapper/                      # MyBatis XML
│   │           └── {도메인}/
│   │               └── {도메인}Mapper.xml
│   └── test/
│       └── java/atms/
├── build/                                   # 빌드 산출물
└── gradle/wrapper/                          # Gradle Wrapper
```

2.3. 2.3 새 도메인 추가 순서

비즈니스 도메인을 추가할 때 아래 순서로 파일 생성:

```
1. atms/{도메인}/mapper/{도메인}Mapper.java        # MyBatis Interface
2. resources/mapper/{도메인}/{도메인}Mapper.xml     # SQL 쿼리
3. atms/{도메인}/dto/{도메인}Request.java           # 요청 DTO
4. atms/{도메인}/dto/{도메인}Response.java          # 응답 DTO
5. atms/{도메인}/service/{도메인}Service.java       # 비즈니스 로직
6. atms/{도메인}/controller/{도메인}Controller.java # REST API
```

2.4. 2.4 주요 설정

application.yml:

```yaml
server:
  port: 8085

spring:
  datasource:
    url: jdbc:h2:mem:testdb   # ← DB 확정 후 변경
    driver-class-name: ...

mybatis:
  mapper-locations: classpath:mapper/**/*.xml
  configuration:
    map-underscore-to-camel-case: true

jwt:
  secret: {시크릿키}
  access-token-expiry: 3600000      # 1시간
  refresh-token-expiry: 86400000    # 24시간
```

2.5. 2.5 빌드 & 실행

```
# 빌드
./gradlew build

# 실행
./gradlew bootRun

# 테스트
./gradlew test

# Swagger UI
http://localhost:8085/swagger-ui.html
```

3. 3. Frontend 구조 (nh-atms-web)

3.1. 3.1 기술 스택

항목	버전	비고
React	19.2.0	
TypeScript	5.9.3	
Vite	7.3.1	빌드 도구
Ant Design	6.3.1	UI 프레임워크
Zustand	5.0.11	상태관리
React Router	7.13.0	라우팅
Axios	1.13.6	HTTP 클라이언트
React Query	5.90.21	서버 상태 관리
dayjs	1.11.x	날짜 처리

3.2. 3.2 디렉토리 구조

```
nh-atms-web/
├── index.html                          # HTML 엔트리
├── package.json                        # 의존성
├── vite.config.ts                      # Vite 설정 (프록시, alias)
├── tsconfig.json                       # TypeScript 설정
├── eslint.config.js                    # ESLint 설정
├── public/                             # 정적 파일
└── src/
    ├── App.tsx                         # 앱 루트 (라우터 설정)
    ├── main.tsx                        # 엔트리포인트
    ├── index.css                       # 글로벌 스타일
    │
    ├── api/                            # API 관련
    │   └── axiosInstance.ts            # ☑ Axios 인스턴스 (토큰 자동관리)
    │
    ├── components/                     # ☑ 공통 컴포넌트 (구현 완료)
    │   ├── SearchForm.tsx              # 동적 조회조건 폼
    │   ├── DataTable.tsx               # 공통 테이블 (엑셀 내보내기)
    │   └── PageTemplate.tsx            # 페이지 기본 레이아웃
    │
    ├── layouts/                        # ☑ 레이아웃 (구현 완료)
    │   ├── AppLayout.tsx               # 전체 레이아웃 (사이드바+헤더+탭+콘텐츠)
    │   ├── AppHeader.tsx               # 상단 헤더
    │   ├── AppSidebar.tsx              # 좌측 메뉴 (현재 하드코딩)
    │   └── AppTabs.tsx                # 탭 네비게이션
    │
    ├── pages/                          # 페이지 (도메인별 폴더)
    │   ├── login/
    │   │   └── LoginPage.tsx           # ☑ 로그인 (구현 완료)
    │   ├── dashboard/
    │   │   └── DashboardPage.tsx       # 대시보드 (스캐폴드)
    │   ├── transaction/
    │   │   └── TransactionSpecificPage.tsx  # 거래조회 (목데이터)
    │   ├── cash/
    │   │   └── CashInquiryPage.tsx     # 시재조회 (목데이터)
    │   └── {도메인}/                   # ← 도메인별 폴더 추가
    │       └── {화면명}Page.tsx
    │
    ├── routes/                         # 라우팅
    │   └── AppRouter.tsx               # 라우트 정의
    │
    ├── stores/                         # Zustand 상태
    │   ├── useAuthStore.ts             # ☑ 인증 상태 (구현 완료)
    │   └── useTabStore.ts              # ☑ 탭 상태 (구현 완료)
    │
    ├── types/                          # 타입 정의
    │   ├── menu.ts                     # 메뉴 타입
    │   └── search.ts                   # 검색조건 타입
    │
```

```
    └── utils/                          # 유틸
        └── excelDownload.ts            # 엑셀 다운로드
```

3.3. 3.3 새 화면 추가 순서

```
1. src/pages/{도메인}/{화면명}Page.tsx  생성
2. src/routes/AppRouter.tsx             라우트 추가
3. src/layouts/AppSidebar.tsx           메뉴 추가 (추후 DB 연동으로 변경)
```

3.4. 3.4 핵심 공통 컴포넌트

SearchForm: 동적 조회조건 폼

- `fields: SearchField[]` 배열로 조회조건 자동 생성
- 지원 타입: `input`, `select`, `datePicker`, `dateRangePicker`
- `onSearch`, `onExcel` 콜백

DataTable: 공통 데이터 그리드

- Ant Design Table 래핑
- 엑셀 내보내기 내장
- 페이징, 정렬

PageTemplate: 페이지 랩퍼

- 제목 표시
- 일관된 페이지 레이아웃

3.5. 3.5 주요 설정

vite.config.ts:

```
export default defineConfig({
  resolve: {
    alias: {
      '@': path.resolve(__dirname, './src'),  // @ → src/
    },
  },
  server: {
    port: 5173,
    proxy: {
      '/api': {
        target: 'http://localhost:8085',
        changeOrigin: true,
      },
    },
  },
});
```

3.6. 3.6 빌드 & 실행

```
# 의존성 설치
npm install

# 개발 서버
npm run dev
# → http://localhost:5173

# 빌드
npm run build
# → dist/ 폴더에 산출물

# 린트
npm run lint
```

---

4. 4. 개발 환경 설정

4.1. 4.1 필수 설치

도구	버전	용도
JDK	21	Backend
Node.js	22.x LTS	Frontend
npm	10.x	패키지 관리
Git	최신	형상관리
IDE	IntelliJ (Backend), VS Code (Frontend)	

4.2. 4.2 실행 순서

```
1. Backend 실행: nh-atms-api/ 에서 ./gradlew bootRun
2. Frontend 실행: nh-atms-web/ 에서 npm run dev
3. 브라우저: http://localhost:5173
4. Swagger: http://localhost:8085/swagger-ui.html
```

4.3. 4.3 Git 브랜치

```
main ← 배포용 (직접 push 금지)
 └── develop ← 통합 개발
      ├── feature/{도메인}-{기능}   ← 기능 개발 브랜치
      └── fix/{이슈설명}           ← 버그 수정 브랜치
```

작업 흐름:

1. `develop`에서 `feature/` 브랜치 생성
2. 작업 → 커밋 → push
3. PR 생성 → 코드 리뷰 → `develop`에 merge

---

5. 5. 현재 구현 상태

5.1. 5.1 Backend

모듈	상태	설명
인증 (auth)	☑ 완료	로그인, 토큰갱신, 로그아웃
Security	☑ 완료	JWT 필터, CORS
공통 응답	☑ 완료	ApiResponse, GlobalExceptionHandler
DB 연결	❌ 미완	H2 인메모리 (DB 미정)
비즈니스 API	❌ 미시작	도메인별 API 구현 필요

5.2. 5.2 Frontend

모듈	상태	설명
로그인	☑ 완료	SSO/FORM 로그인
레이아웃	☑ 완료	사이드바, 헤더, 탭
공통 컴포넌트	☑ 완료	SearchForm, DataTable, PageTemplate
인증 상태관리	☑ 완료	Zustand + sessionStorage
메뉴 (동적)	❌ 미완	현재 하드코딩, DB 연동 필요
비즈니스 화면	❌ 미시작	mock 데이터만 있음

5.3. 5.3 우선 구현 순서

```
1단계: 시스템관리 (메뉴, 권한, 사용자, 공통코드)
       → 동적 메뉴 로딩 → 다른 화면 개발 기반

2단계: 기기관리 (본지점, 코너, 기기정보)
       → 마스터 데이터 조회 기반

3단계: 시재관리 (거래내역, 투입, 기기시재)
       → 핵심 업무

4단계: 장애관리, 마감처리
       → 핵심 업무

5단계: 저널관리, 원격관리, 통계/보고서, 대시보드
       → 부가 기능
```

---

6. 6. 참고 문서

문서	위치	설명
코딩컨벤션	`2.개발문서/01_코딩컨벤션.md`	코딩 규칙
API 스펙	`2.개발문서/02_API스펙정의.md`	API 설계
작업 명세서	`2.개발문서/03_화면작업명세서_템플릿.md`	작업 배분 템플릿
DB 스키마	`2.개발문서/04_DB스키마설계.md`	DB 테이블 설계
AS-IS 분석	`0.기본자료/농협은행/NH AS-IS메뉴 분석.md`	전체 메뉴 분석
TO-BE 메뉴	위 파일 내 TO-BE 섹션	목표 메뉴 구조